

H1 软件系统设计 2025 春 - 设计模式作业反馈

写在前面

先和大家说一声抱歉，由于助教人手有限，没有办法对每个小组的设计逐个进行点评。我们认真看完了所有同学三次迭代的文档，对一些共性的问题在本文档中进行指出，同时也分享一下多个小组有亮点的设计。虽然这门课程期末已经结束，但是希望同学们能认真阅读本文，看看自己小组的设计还有哪些问题，或是有哪些好的设计可以应用到未来的项目中。

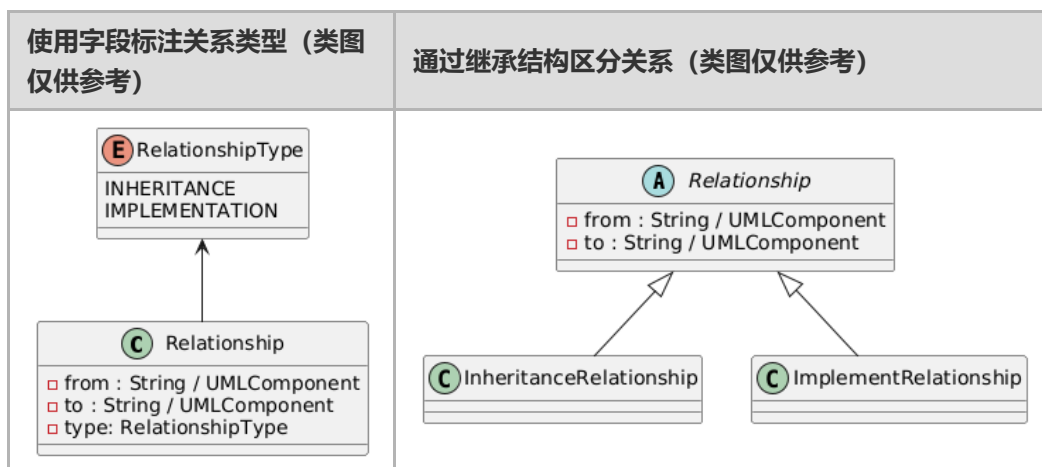
H2 迭代一

H3 类图元素的封装

在这个迭代中，我们要求同学们实现一个读取单个Java文件并生成PlantUML类图的工具。我们要生成的 `ClassDiagram` 类是一个很复杂的对象，里面包含了**类和接口**，类中还包含了**方法和字段**，每个方法中还包含方法名、返回值类型、参数类型等信息。因此在迭代一中，你最重要的任务是完成对上述加粗内容的**封装**。你应当实现形如下面的类：`ClassInfo`、`InterfaceInfo`、`MethodInfo`、`FieldInfo` 等等（当然，类名不做要求）。

此外，要实现一个类图生成器，你也应当封装类图中的关系边 `Relationship`。在批改作业时，我们发现一部分小组直接在 `ClassInfo` 或 `InterfaceInfo` 中保存相连类图元素的名称，这种做法的可扩展性要稍差一些。（比如添加新的关系类型，必须修改 `ClassInfo` 类）

对于关系类型的处理，下面两种方法以及其他相似的方法我们均在这个小项中给了满分：



部分小组直接使用字符串 `String` 类存储类图元素或关系，这种设计会导致后续迭代（更多元素、分析器、命令行工具）实现困难，不是良好的设计，我们会扣除一定的分数。

H3 设计模式使用

在迭代一中，我们不要求小组一定要采用什么设计模式，即使没有任何设计模式也能拿到满分。我们在这里列举一些小组中合适的设计模式供大家参考：

H4 组合模式

类图 `ClassDiagram` 是一个多层级的结构，可以让 `ClassDiagram`、`ClassInfo`、`InterfaceInfo`、`MethodInfo`、`FieldInfo` 等类都实现 `UMLComponent` 接口，并由每个类实现 `generateUML()` 方法。

H4 建造者模式

在迭代一中，`ClassInfo` 对象的创建非常复杂，需要指定类名、字段列表、方法列表等内容。因此，可以使用建造者模式来逐步构建 `ClassInfo` 对象。

```
public class ClassInfoBuilder {
    public void setClassName() { ... }
    public void addField(FieldInfo fieldInfo) { ... }
    public void addMethod(MethodInfo methodInfo) { ... }
    public ClassInfo build() { ... }
}
```

H4 （简单）工厂模式

由专门的工厂类来负责创建 `ClassInfo`、`InterfaceInfo`、`MethodInfo`、`FieldInfo` 等对象。

H4 访问者模式

- 虽然访问者模式(Visitor)不在课程授课范围内，但是大家编译原理课都学过了，事实证明学得还挺好的（
- 如果你还不知道什么是访问者模式，我们推荐你阅读 [访问者设计模式](#)。

H5 用法1：遍历抽象语法树 AST

利用 Visitor 模式遍历由JavaParser生成的抽象语法树，获取对应的类、接口、字段、方法信息，并保存起来。

H5 用法2：支持不同的文本类图格式

一部分小组考虑了生成类图阶段的可扩展性，除了PlantUML文本，还有其他的可以描述类图结构的文本结构。在上面的组合模式实现中，如果要新增一种 Mermaid 文本的需求，需要在每个实现了 `UMLComponent` 接口的类中都新增一个 `generateMermaid()` 方法，这**显然不符合开闭原则**。

因此可以使用访问者模式，让这些类都对外提供一个 `accept(visitor: Visitor)` 方法，由具体的 Visitor 类来遍历具有层次结构的类图，并生成对应的文本信息。

H3 设计存在的问题

在阅读大家的文档/代码时，我们发现部分小组会将多个类写在一个文件里。这种不分文件的编码习惯非常糟糕，使得代码可维护性、可扩展性都很差。

H2 迭代二

在迭代二中，我们新增了抽象类（仅加一个 `isAbstract` 的字段即可）、枚举类等类图元素，在这里如果迭代一有良好的封装，应该不会太难。

H3 嵌套类型处理

这个和设计模式无关，但是是一个实现上的小难点。有的小组使用字符串处理，去找到 `<` 符号所在的位置，再分割字符串，实现起来难度较大。更简单的实现方式是直接用Visitor模式去遍历JavaParser解析出的 `Type` 类或是自行递归地进行处理。

注：这里的实现不作为评分点

H3 类型识别

需要注意的是，`void`、`Integer`、`ArrayList` 等 Java 内置类型和内置类是不需要单独绘制在类图中的。打表来排除这个内置类型是及其困难的，一个可行的实现方式是用哈希表来记录 输入文件 中出现的类名/接口名，没在哈希表中的类名/接口名（比如：`ArrayList`、`Queue`）**不需要绘制关系**。

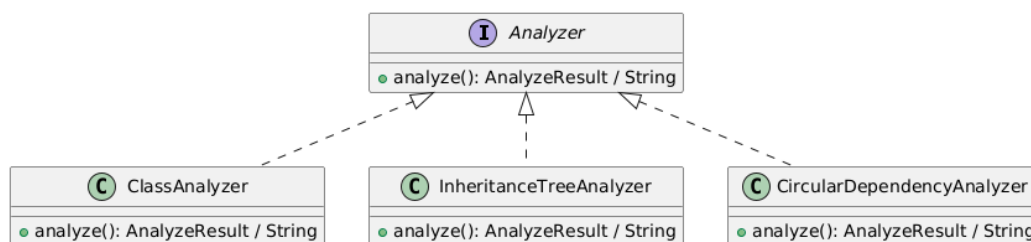
H3 分析器设计

分析器设计将作为迭代二的重点评分项，你的设计在添加新的分析器时应能较好地满足开闭原则。

参考设计如下：

- 有一个 `CodeSmellAnalyzer` 抽象类或接口
- 有 `ClassAnalyzer`、`InheritanceTreeAnalyzer`、`CircularDependencyAnalyzer` 等具体的分析器实现类，并实现了 `CodeSmellAnalyzer` 接口/继承了 `CodeSmellAnalyzer` 抽象类

类图大概长这个样子：



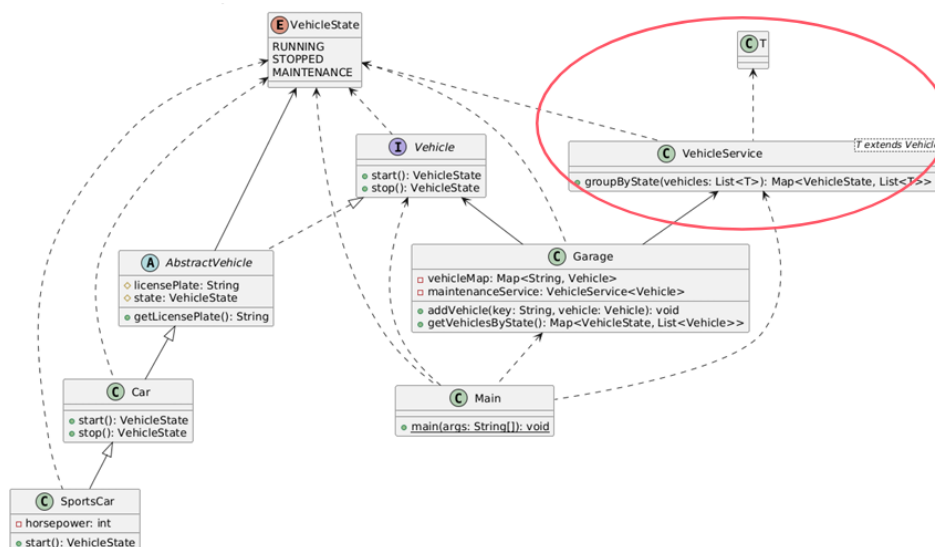
有的小组还在这里分析器模块用了 责任链模式 / 模板方法模式 / 组合模式，仅供参考。

责任链模式	模板方法模式	组合模式
多个分析器视为一个分析器链，每个分析器持有一个 <code>nextAnalyzer</code> 。	分析器父类中定义了预处理、分析的方法，由子类具体实现逻辑。	类分析器中还持有：过大类分析、过小类分析、数据类分析等分析器。

对于继承树分析和循环依赖分析，你需要构建继承树和依赖图的数据结构。循环依赖检测只要求找到图中的一个环即可，如果想了解更多关于强连通分量的知识，可以学习 Tarjan 算法。

H3 文档存在的问题

- 有一部分小组在 为测试用例中的 `Vehicle.java` 文件生成类图中出现了问题。这个测试用例包含泛型参数 `T`，一些小组绘制的类图中出现了一个具体的类依赖 `T` 的边。
`T` 是一个泛型参数不是一个类，这样的类图会被扣掉少量分数。



扣的分很少，因为 OJ 测试用例也没考虑到同学们会犯这个小错误。

- 还有极少数组理解错了文档模板的要求，把 PlantUML 文本截了一张图交了上来，而不是按要求提交 "PlantUML 文本生成的类图图片（仅图片）"。很遗憾，我们只能扣除类图一部分的分数。
- 我们要求提交 PDF 格式，请各位同学一定要检查 PDF 中是否正确包含了图片，以及图片是否清晰。我们发现极少部分小组存在下面的问题：

- 提交成Markdown文件，同时里面的图片不是在线链接，助教看不到你的类图
- 提交PDF文件，但是类图过于模糊，实在是啥也看不清TAT。可以考虑清晰一点的截图、使用SVG格式的图片或者将类图分开来画。

H3 设计存在的问题

还是有小组不分文件呜呜，只能狠心扣分了。

一般来说，**Java项目一个类一个文件**，内部类除外。

H2 迭代三

H3 命令行工具实现

H4 命令模式

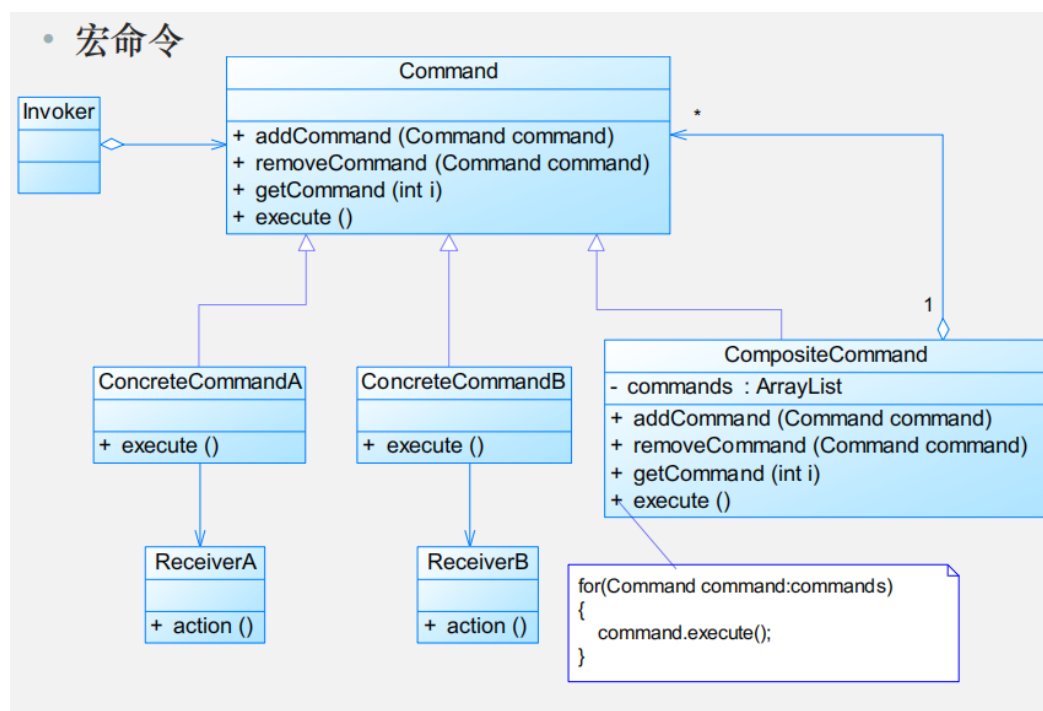
在前两次迭代中，我们没有刻意让同学们用一些设计模式，这比较符合真实的项目需求。在迭代三中，我们其实暗示了要使用**命令模式**来完成命令行工具的实现（因为有 `undo` 的要求）。

要求：

- 有 `Command` 抽象类或者接口，接口有 `execute()` 和 `undo()` 两个方法。额外设计一个 `UndoableCommand` 接口也是可行的设计。
- 有多个具体的 `Command` 类，封装了具体的操作。
- 能自行实现或利用库完成对命令行的解析，并创建不同的命令对象。

每个具体的命令类实现 `undo()` 方法，例如Add命令的对应的undo方法中，应当将刚刚添加的内容删除掉。将所有执行过的可以撤销的命令压到一个栈中，每次撤销操作就弹出栈顶命令 `Command`，并调用对应的undo方法。

对于修改字段/方法命令，除了可以沿用上面的设计外，还可以使用支持宏命令的命令模式。



- 宏命令又称为组合命令，它是命令模式和组合模式联用的产物。
- 宏命令也是一个具体命令，不过它包含了对其他命令对象的引用，在调用宏命令的 `execute()` 方法时，将 **递归调用** 它所包含的每个成员命令的 `execute()` 方法。

在这种设计中，修改字段命令包含一个 `DeleteFieldCommand` 和 `AddFieldCommand`，调用 `execute()` 方法时，先调用 `DeleteFieldCommand` 和 `AddFieldCommand` 的 `execute()` 方法；在调用 `undo()` 方法时，**倒序调用** `DeleteFieldCommand` 和 `AddFieldCommand` 的 `undo()` 方法。

H4 撤销操作：命令模式 VS 备忘录模式

一些小组使用原型模式创建多个 `ClassDiagram` 的深拷贝，用来实现撤销操作。每次撤销，就回到上一个拷贝的 `ClassDiagram`。这种设计实际是**备忘录模式**。

助教团队认为，在JClassDiagram迭代三中，**带撤销的命令模式 优于 备忘录模式**。

带撤销的命令模式的优势在于，每个具体命令类中保存了类图每次的变化量，内存开销小，而原型模式 + 备忘录模式每次都要创建 `ClassDiagram` 的深拷贝，内存开销较大。

那什么时候用备忘录模式呢？（当然这里也能用）例如PhotoShop的合并图层操作，想要完成撤销就只能使用备忘录模式，拷贝一份之前的状态，因为**合并图层操作是不可逆的**。

H3 配置解析

很多同学忽略了实验文档中一句重要的话：

你需要为ClassDiagram类实现一个 `loadConfig(configFile: String)` 的公共方法，该方法接受一个字符串参数，表示配置文件的路径。**虽然这是我们实验的最后一个任务，但你仍然需要考虑弹性的设计。你需要考虑使用JSON格式、YAML格式的配置文件的情况，但为了减轻工作量，你不用具体实现它们。**

能否应对配置文件的可扩展性是一个得分点，一个设计供大家参考，这种设计符合**策略模式**：

```
config/
├─ Config.java           # 配置类
├─ ConfigLoader.java     # 配置加载器接口
├─ ConfigLoaderFactory.java # 配置加载器工厂，根据扩展名创建对应的配置加载器
└─ XmlConfigLoader.java  # XML格式配置加载器
```

H3 分析器创建

在迭代三中，实验要求根据配置文件中的内容，选择开启一些分析器。一个好的设计是，读取配置内容，并根据配置内容创建对应的分析器对象。

- 可以实现一个分析器简单工厂，根据分析器名字创建对应对象，这里还可以用 **表** 来简化设计。
- 也有小组利用Java的反射机制来创建分析器对象，这也是可行的。

H3 文档和设计存在的问题

存在的问题在迭代一迭代二中基本写过了：

- 没有分多文件实现
- 提交的PDF不符合规范

但也存在一些新的问题，例如文档前后描述矛盾，并在翻阅GradeScope上的源代码时，发现代码中的设计和文档描述的不一致。大家用DeepSeek、ChatGPT生成文档的时候要谨慎。

以上就是助教团队在批改作业时发现的问题，文档发出时大家期末考应该也考完了，希望大家都能取得优异的成绩。如果对设计模式部分的课程实验有什么建议，欢迎联系助教。

2025 春 实验文档和实验框架代码已迁移到 GitHub：

- Docs: <https://github.com/Nboxff/nju-ssd-2025-design-pattern-docs>
- Framework: <https://github.com/Nboxff/nju-ssd-2025-design-pattern-lab>